

LESSON GUIDE

Technology Plus

Unit 2: Programming

4.1.1 – Programming Language Categories

Lesson Overview:

In this lesson, students will learn about the four main categories of programming languages and be able to identify when one is preferred over another for problem solving.

Students will:

- Differentiate programming language categories based on their characteristics.
- Categorize programming languages based on the types of problems they are used to solve.
- Choose an appropriate solution for a given problem.

Guiding Question: How do developers choose a programming language based on their project requirements?

Suggested Grade Levels: 8-12

Technology Needed: Yes

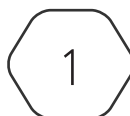
Approximate Time Required: 65-80 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 4.1 – Compare and contrast programming language categories.
 - Interpreted
 - Scripting languages
 - Markup languages
 - Compiled programming languages
 - Query languages
 - Assembly languages

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



CYBER.ORG
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center
All Rights Reserved. Not for Distribution.

Lesson Background

Background Information:

Programming languages have grown dramatically over time, from the first simple programs designed for early giant computers like the ENIAC to the modern-day code running on devices as small as our watches.

This lesson introduces students to four categories of programming languages while emphasizing the importance of computational thinking and problem-solving over memorizing how many languages you know. Unlike human languages, which serve many purposes, programming languages all share the same goal: turning human instructions into something a computer can execute. By understanding these categories, students will be better equipped to choose the right type of language for different programming challenges.

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 4.1.1 – Programming Language Categories • Lab Slides 4.1.1 – Language Lookup
Materials per Student	• Student Notes 4.1.1 – Programming Language Categories • Student Handout 4.1.1 – Programming Language Categories • Assessment 4.1.1 – Programming Language Categories (optional)

Cyber Terms:

Term	Definition
abstraction	simplification through the removal of unnecessary details so only the essential components remain
assembly	low-level programming languages that provide a symbolic representation of a computer's machine code instructions
compiled	programming languages that are transformed into machine code by a compiler before execution
interpreted	programming languages that are executed directly by an interpreter, which reads and executes code line by line at runtime
paradigm	a style or way of programming
query (language)	specialized languages designed for querying and manipulating data in databases

4.1.1 – Programming Language Categories

Guiding Question: How do developers choose a programming language based on their project requirements?

Stop to Talk About Octal (15-20 minutes)

- Ask students to think about the ways in which they communicate with others. When do they use electronic communication? When do they use spoken language? When do they use nonverbal language?

Students may share that they use electronic communication when they are talking to people physically far away from them, spoken language when they are with the person, and nonverbal language when they can see each other but cannot hear each other, like in a crowded space.
- Today's lesson is going to explore how different categories of programming languages are used to communicate with our devices, and how developers know which type of language works best for a given problem.

Purpose-Driven Language Choices (35-40 minutes)

- Distribute Student Handout 4.1.1 – Programming Language Categories and encourage students to fill in the answers during the lesson.

Abstraction

- In programming languages, **abstraction** removes details from human view to make it easier to read and write code. Each level of abstraction simplifies more complexities and provides more scaffolding for the user.
- Use the car analogy, with images on Slide 3, to explain each level.
- **Low-level** abstraction means users are interacting closely with machine instructions and need to understand machine code and/or assembly language. This is like building the car yourself and working directly with the engine.
- **Mid-level** languages can interact directly with hardware, and they offer user-friendly environments. This is like driving a manual transmission car – you control gear shifting, but not every physical detail.
- **High-level** means there is a lot of abstraction to hide most of the details from the user. Users work in environments that are easy to understand and easy to write in, with the language itself handling interaction with machine code. This is like driving an automatic transmission car – you only manage the pedals and the steering, while the car handles mechanics.
- **Very high-level**, also known as domain-specific, focuses on declaring what we want to do without needing to know how to do it. This final example is like hailing a taxi or ride-share – you input where you want to go and everything else is handled for you.

Programming Languages by Level of Abstraction

Category	Description	Examples
Low-level	Very close to the computer's hardware. Hard for humans to read, but extremely fast and efficient.	Machine code (1s and 0s), Assembly
Mid-level	Balances between hardware control and readability. Still requires more technical knowledge, but more human-friendly than assembly.	C, C++
High-level	Easier for humans to read and write. Hides most hardware details so programmers can focus on logic and problem-solving. Some languages in this category (e.g., Java, C#) can be compiled or interpreted depending on environment.	Java, Python, C#, JavaScript, PHP, Perl
Very High-level (Domain-Specific)	Designed for a special purpose. You can describe what you want, and the language handles how it gets done.	SQL, LDAP, HTML, XML, Markdown, Scratch (block-based)

Assembly

- **Assembly** language is a low-level programming language that provides a close representation of machine code to interact directly with hardware.
- Used in system programming, embedded systems, and performance-critical applications
- Specific to a CPU architecture, not portable across different systems
- Examples:
 - x86 Assembly: used for Intel and AMD processors, common in desktops, servers, and gaming consoles
 - ARM Assembly: used for ARM processors, common in mobile devices and embedded systems
 - MIPS Assembly: used for MIPS processors, common in educational contexts and some embedded systems
- *Advantages:*
 - Highly optimized code that can directly control hardware
 - Fast and efficient execution

- *Disadvantages:*
 - Difficult to read and write
 - Not portable across different hardware
 - Slow development and debugging

Compiled

- **Compiled** language is high- or mid-level programming languages that require the use of a compiler to translate source code into machine code.
 1. Write the application (source code) in a programming language
 2. Use a compiler to translate the source code to machine code
 3. Execute the program
- Often use integrated development environments (IDEs) which embed compilers
- Examples:
 - Java, C, C++, C#
- *Advantages:*
 - Better handling of large-scale applications
- *Disadvantages:*
 - Need to recompile after all changes

Interpreted

- **Interpreted** language is high-level programming languages where each line of source code is executed by an interpreter.
- Examples:
 - Scripting languages are designed primarily for automating tasks, controlling other software, or operating in specialized scripting environments.
 - Typically embedded in other systems such as web browsers and server environments
 - Easier to use than general-purpose languages with less formal structure
 - Are often used when cross-platform deployment is needed, because the same code can run on different operating systems if an interpreter exists
 - Shell scripting is used for automating system tasks and command-line operations.
 - Examples: Bash (Linux), PowerShell (Windows)
 - Web scripting adds interactivity and automation in web environments
 - Examples: JavaScript, PHP

- Task automation and general scripting help to automate repetitive tasks or integrate software tools.
 - Examples: Python, Perl
- Markup languages are designed to format, annotate, and structure documents rather than performing general computational tasks.
 - Consist of tags or annotations that describe how content should be displayed or interpreted
 - Easy to learn and use for document formatting purposes
 - Cannot perform complex computations or logic
 - Web structuring: HTML (HyperText Markup Language)
 - Data structuring: XML (eXtensible Markup Language)
 - Text structuring: Markdown
- *Advantages:*
 - Easier to debug during development
 - Rapid development
- *Disadvantages:*
 - Slower execution than compiled

Functional Programming

- A programming **paradigm** that emphasizes the creation and application of functions to build programs.
- Can be compiled or interpreted languages.

Object-oriented Programming

- A programming paradigm that emphasizes the creation of classes, complete with attributes and methods, to model real-world objects.
- Can be compiled or interpreted languages.

Query

- **Query language** is domain-specific programming languages designed to be question-based within databases.
- Examples:
 - Structured Query Language (SQL)
 - Lightweight Directory Access Protocol (LDAP)
- *Advantages:*

- Easy to use
- Efficient in handling complex queries and large datasets
- Declarative nature makes it easier to write and understand
- *Disadvantages:*
 - Limited in scope
 - Not suitable for general-purpose programming
 - Performance variability based on underlying database engine and implementations

Lab: 4.1.1 Language Lookup (25-30 minutes)

- This lab requires access to the CYBER.ORG Range, the Windows Server 22 Machine, and Lab Slides 4.1.1 – Language Lookup.
- The lab can be performed as a class activity or an independent student assignment.
- Introduce the guiding question and objectives for the lab.
- Students should record responses to reflection prompts in their Student Handout.
- Remind students about adding “.py” and “.cpp” to file names to load the correct language libraries for that file.
- Emphasize the importance of careful typing to reduce errors.
- While this lesson covered Assembly, Compiled, Interpreted, and Query languages, the lab will focus solely on Compiled and Interpreted. Understanding how to write code in Assembly is beyond the scope of this course and Query will be covered in depth in Unit 6.
- Students create a short program in C++ and Python that will sum two integers and print the result.
- Students will compare the two programs and discuss differences.
- Decide if the next section will be done by the whole class or in small groups.
- If choosing the whole class, display the examples one-by-one and discuss answers after providing thinking time. Slides 11-14 are animated with answers and sample code.
- If choosing small groups, consider printing slides 11-14 as a handout for groups to record their responses on, then share the solutions after every group has finished.
- Explain the octal numbering system.
 - Slide 11 – Assembly languages would best solve this problem because it requires direct hardware control and fast execution. The sample code is written in ASM.
 - Slide 12 – Interpreted languages would best solve this problem because it requires cross-platform deployment. The sample code is written in Python.
 - Slide 13 – Query languages would best solve this problem because it requires searching and retrieving data from a database. The sample code is written in SQL.

- Slide 14 – Compiled languages would best solve this problem because it requires speed and efficiency in execution to properly handle the calculations and graphics rendering. The sample code is written in C++.

Putting It All Together

- Which program felt faster to write and why?
Students will most likely say Python since it required less executable lines of code.
- Which type of programming language would you use if programming a pedometer with limited memory and no operating system?

Assembly languages would be most appropriate to handle the low-level access (hardware) and resource constraints.

Putting It All Together (10 minutes)

- Describe the levels of abstraction.
Low-level abstraction means communicating directly, or nearly directly, with hardware. Few details are removed, and users must understand machine code. High-level is when most details have been hidden from the user, allowing the user to easily communicate with the computer.
- Which programming language category is best for writing code that directly accesses the computer's hardware?

Assembly languages allow direct interaction with hardware.

- Which programming language category is executed line-by-line, making it run slowly but allowing for faster debugging?

Interpreted languages.